

Lab03_Oliveira

AUTHOR

Geovana de Oliveira Barros da Silva - 175367

Laboratório 3

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.
- Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;

IMPORTAÇÃO DE AIRPORTS

```
library(readr)

myaero = cols_only(
  IATA_CODE = 'c',
  CITY = 'c',
  STATE = 'c',
  LATITUDE = 'd',
  LONGITUDE = 'd')

airports = read_csv("airports.csv", col_types = myaero)
```

- Para o arquivo de vôos:
 - a. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;
 - b. Leia apenas 1 milhão de linhas por vez;
- c. Remova vôos em que o aeroporto de destino comece com a letra 1;
- d. Remova registros em que existam pelo menos uma coluna faltante;
- e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
- f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

IMPORTAÇÃO DE FLIGHTS

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
getStats = function(input, pos){
  #Remover todos destinos que começa com 1
  input |> filter(!grepl("^1", DESTINATION_AIRPORT )) |>
  #Remover todas as colunas sem registros
  filter(!is.na(DESTINATION_AIRPORT), !is.na(ARRIVAL_DELAY)) |>
  group_by(DESTINATION_AIRPORT) |>
  summarise(
    soma_atraso = sum(ARRIVAL_DELAY),
    num_voos = n(),
    .groups = "drop"
  )}

myfli = cols_only(
  DESTINATION_AIRPORT = 'c',
  ARRIVAL_DELAY = 'd'
)

flights = read_csv_chunked("flights.csv",
  callback = DataFrameCallback$new(getStats),
  chunk_size = 1000000,
  col_types = myfli)
```

```
estat_suficientes = flights |> group_by(DESTINATION_AIRPORT) |>
  summarise(
    media_atraso = sum(soma_atraso)/ sum(num_voos),
    .groups = "drop")
```

IMPORTAÇÃO DE AIRLINES

```
airlines = read_csv("airlines.csv")
```

Rows: 14 Columns: 2

— Column specification —————

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

2. Selecione a operação apropriada join para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

a. Identifique a coluna que é a chave na tabela `flights`;

```
flights
```

```
# A tibble: 1,874 × 3
  DESTINATION_AIRPORT soma_atraso num_voos
  <chr>                <dbl>    <int>
1 ABE                  3413      350
2 ABI                  5926      446
3 ABQ                 20448     3241
4 ABR                  -645      124
5 ABY                 1339      169
6 ACT                 4436      271
7 ACV                 2748      294
8 ACY                 9069      701
9 ADK                  -293       19
10 ADQ                  72       59
# i 1,864 more rows
```

Para identificação de coluna que é chave, basta identificar colunas onde observações são iguais em ambas tabelas - É DESTINATION_AIRPORTS

b. Identifique a coluna que é a chave na tabela airports;

```
airports
```

```
# A tibble: 322 × 5
  IATA_CODE CITY        STATE LATITUDE LONGITUDE
  <chr>      <chr>      <chr>    <dbl>    <dbl>
1 ABE      Allentown    PA        40.7     -75.4
2 ABI      Abilene       TX        32.4     -99.7
3 ABQ      Albuquerque  NM        35.0    -107.
4 ABR      Aberdeen     SD        45.4     -98.4
5 ABY      Albany       GA        31.5     -84.2
6 ACK      Nantucket    MA        41.3     -70.1
7 ACT      Waco         TX        31.6     -97.2
8 ACV      Arcata/Eureka CA        41.0    -124.
9 ACY      Atlantic City NJ        39.5     -74.6
10 ADK      Adak         AK        51.9    -177.
# i 312 more rows
```

Na tabela airports é IATA_CODE

c. Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports?

```
aeroportos_ausentes <- estat_suficientes |> anti_join(airports, by = c("DESTINATION_AIRPORT" =
aeroportos_ausentes
```

```
# A tibble: 0 × 2
# i 2 variables: DESTINATION_AIRPORT <chr>, media_atraso <dbl>
```

d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
e. Armazene a tabela resultante no objeto flights.

```
tabela <- estat_suficientes |>
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

flights <- tabela
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
library(dplyr)

airports_state = airports |> filter(!is.na(STATE)) |> group_by(STATE) |>
  summarise(
    qtd_aerportos = n(),
    .groups = "drop") |>
  arrange(desc(qtd_aerportos))

airports_state
```

```
# A tibble: 54 × 2
  STATE qtd_aerportos
  <chr>      <int>
1 TX          24
2 CA          22
3 AK          19
4 FL          17
5 MI          15
6 NY          14
7 CO          10
8 MN           8
9 MT           8
10 NC           8
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

- Carregue o pacote leaflet;
- Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico;
- Armazene o grafico em b) numa variável chamada this;
- Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

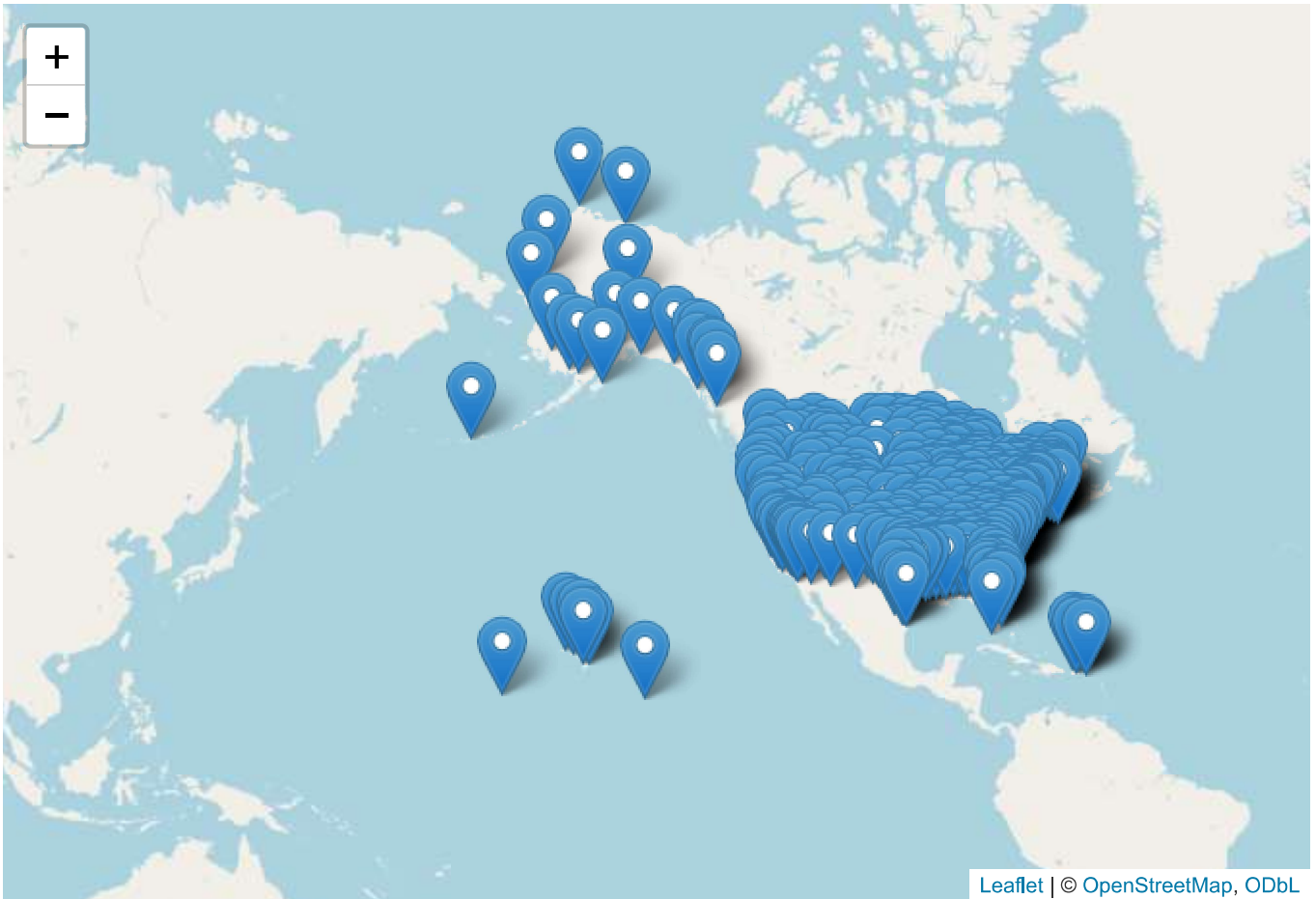
```
library(leaflet)
library(dplyr)

this <- leaflet(flights) |> addTiles() |>
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~ paste(
      CITY,
```

```
round(media_atraso,2))  
)
```

Warning in validateCoords(lng, lat, funcName): Data contains 3 rows with either missing or invalid lat/lon values and will be ignored

this



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 23:14:19 -03"
```